

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

FACULTAD DE INGENIERÍA

ESCUELA DE CIENCIAS Y SISTEMAS

INTRODUCCIÓN A LA PROGRAMACIÓN Y COMPUTACIÓN 2 (IPC2)

MODELADO ARQUITECTÓNICO, ESTRUCTURAS DE DATOS PROPIAS Y ALGORITMOS DE BÚSQUEDA DE CAMINOS EN MALLAS BIDIMENSIONALES DINÁMICAS

Ensayo Académico de Diseño e Implementación Software para el Sistema "Chapín Warriors"

- **Estudiante:** Erick Adrián Patzán Pérez
- **Carné:** 202502375
- **Cátedra:** Introducción a la Programación y Computación 2
- **Sección:** D
- **Tutor / Auxiliar:** Javier Marías

I. INTRODUCCIÓN

En el campo de la ingeniería de software y la ciencia de la computación, el diseño eficiente de algoritmos y la estructuración rigurosa de la memoria constituyen los pilares fundamentales para la resolución de problemas complejos. El desarrollo de sistemas autónomos de navegación en entornos hostiles o en malla plantea un reto dual: por un lado, se requiere una representación abstracta que modele fielmente la topología del terreno y sus restricciones dinámicas; por otro lado, se exige el cómputo de rutas óptimas o viables en presencia de obstáculos fijos, entidades hostiles con capacidades de combate y objetivos específicos de rescate o extracción.

El presente documento expone la fundamentación teórica, el diseño arquitectónico y la implementación práctica del sistema de control para unidades robóticas autónomas denominadas *ChapinEyes*, desarrollado para la empresa ficticia *Chapín Warriors, S.A.* En el marco del curso Introducción a la Programación y Computación 2 (IPC2) de la Facultad de Ingeniería de la Universidad de San Carlos de Guatemala (USAC), este proyecto aborda la problemática de exploración táctica mediante el uso estricto del paradigma Orientado a Objetos (POO) y la construcción artesanal de Tipos de Datos Abstractos (TDA) dinámicos, prescindiendo por completo de las bibliotecas de colecciones preconstruidas que provee el marco de trabajo .NET.

El objetivo general de este ensayo es analizar en profundidad las decisiones de diseño de software adoptadas para resolver el problema de exploración y navegación táctica. Se profundiza en el modelado conceptual de las mallas ortogonales bidimensionales, la abstracción de agentes autónomos y misiones, el comportamiento de las listas simplemente enlazadas y colas FIFO desarrolladas a nivel de punteros, y la formulación sintáctica de algoritmos de Búsqueda en Anchura (BFS). Asimismo, se diferencia formalmente el recorrido BFS estándar para operaciones de rescate civil frente al recorrido BFS "con estado de capacidad de combate" para misiones de extracción de recursos, demostrando cómo la evolución del estado interno del agente modifica el espacio de búsqueda y exige mecanismos eficientes de poda y reconstrucción de rutas. Finalmente, se detalla la integración con la herramienta de visualización vectorial Graphviz para la representación topológica de los resultados.

II. DESARROLLO

2.1. Contextualización del Problema y Abstracción del Dominio

El problema planteado por *Chapín Warriors, S.A.* se sitúa en un escenario de supervisión y respuesta táctica sobre zonas urbanas en conflicto. El entorno espacial se representa como una matriz ortogonal de celdas bidimensionales donde conviven diversos elementos operacionales: puntos de entrada seguros, caminos transitables, zonas intransitables, unidades militares enemigas con valores específicos de capacidad defensiva, unidades civiles vulnerables y recursos estratégicos objeto de extracción.

Para abordar operacionalmente este escenario, se definen dos tipologías de agentes robóticos con capacidades diferenciadas:

- **ChapinRescue:** Agente especializado en misiones de evacuación civil. Su comportamiento está rígidamente acotado a la neutralidad táctica: entra por cualquier punto de acceso transitable, navega estrictamente por vías libres (evitando cualquier conflagración armada o interferencia con unidades militares) y alcanza la posición de la unidad civil para llevar a cabo el rescate. Su paso por celdas con unidades militares o recursos no objetivo está estrictamente prohibido.
- **ChapinFighter:** Agente táctico provisto de un atributo cuantitativo denominado **CapacidadCombate (\$C_R\$)**. Su propósito es penetrar en la malla, sortear o enfrentar unidades militares enemigas y extraer un recurso estratégico. Si en su trayectoria encuentra una unidad militar con capacidad C_U , el robot puede avanzar a través de esa celda únicamente si su capacidad actual es estrictamente superior a la de la unidad ($C_R > C_U$). Al vencerla, el agente sufre un desgaste equivalente, actualizando su capacidad remanente a $C'_R = C_R - C_U$.

La complejidad del sistema radica en que la entrada de datos se realiza mediante la ingesta dinámica de múltiples archivos XML de configuración, los cuales pueden aportar nueva información topológica o actualizar ciudades y robots existentes. Ante la solicitud de una misión por parte del operador, el sistema debe determinar si existe al menos una ruta válida y, de ser así, presentarla visualmente destacando el camino óptimo o declarar formalmente la condición de "Misión Imposible".

2.2. Arquitectura de Software y Modelado Orientado a Objetos

Para garantizar una clara separación de responsabilidades, alta cohesión y bajo acoplamiento, el sistema se estructuró rigurosamente bajo una arquitectura en tres capas: **Modelos, Tipos de Datos Abstractos (TDA) y Servicios**, interactuando con una capa superior de Interfaz de Usuario implementada en WPF (Windows Presentation Foundation).

La arquitectura de clases del dominio se fundamenta en los principios esenciales de la POO:

- **Abstracción y Herencia:** La jerarquía de agentes se encabeza con la clase abstracta **Robot**, la cual encapsula las propiedades comunes **Nombre** y **Tipo**. De esta derivan **RobotRescue** (que restringe su tipo a rescate) y **RobotFighter** (que extiende el modelo incluyendo la propiedad entera **CapacidadCombate**). Esto permite al motor de cálculo tratar polimórficamente a las entidades robóticas mientras aplica reglas de negocio diferenciadas.

- **Encapsulamiento y Composición:** La entidad **Ciudad** actúa como el agregador principal del terreno. Contiene el nombre de la metrópoli, sus dimensiones topológicas (**Filas** y **Columnas**), una instancia de **MatrizOrtogonal** que constituye la representación física del mapa, y una **Lista<UnidadMilitar>** propia. Proporciona métodos de consulta especializada como **TieneCiviles()**, **TieneRecursos()**, **ObtenerPuntosEntrada()**, **ObtenerCiviles()** y **ObtenerRecursos()**.
- **Modulación de Misiones:** La clase **Mision** modela la solicitud del usuario, vinculando el enum **TipoMision** (Rescate o Extracción), la **Ciudad** seleccionada, el **Robot** asignado y las coordenadas destino del objetivo.

Capa	Componentes Clave	Responsabilidad Principal
TDA	Lista<T> , Cola<T> , MatrizOrtogonal , Nodo<T> , NodoMatriz	Gestión dinámica de memoria mediante punteros explícitos sin colecciones de .NET.
Modelos	Robot , RobotRescue , RobotFighter , Ciudad , Celda , Mision	Representación del dominio, reglas de entidad y encapsulamiento de estado.
Servicios	LectorXML , SistemaControl , BuscadorCaminos , GeneradorGraphviz	Orquestación de la lógica de negocio, parseo, resolución de grafos y renderizado.

2.3. Tipos de Datos Abstractos (TDA) Personalizados y Manejo de Memoria Dinámica

Una restricción medular del proyecto radica en la prohibición explícita de utilizar las estructuras genéricas de .NET (**System.Collections.Generic**). En su lugar, toda la gestión de memoria se diseñó manualmente a partir de punteros y asignación de nodos en el Heap. Las estructuras propias implementadas son:

1. Lista Simplemente Enlazada Genérica (**Lista<T>**):

Formada por nodos genéricos **Nodo<T>** (poseedores del valor **Dato** y la referencia **Siguiente**). Ofrece operaciones de inserción **Agregar(T)**, eliminación por índice **Eliminar(int)**, consulta por posición **Obtener(int)** y verificación de estado. Para posibilitar la actualización dinámica exigida por la ingesta XML (donde redefinir una ciudad o robot implica sobrescribir el registro previo), se incorporaron dos métodos avanzados:

- **BuscarIndice(Predicate<T> criterio):** Recorre linealmente la lista evaluando una expresión delegada para retornar la posición exacta del elemento coincidente.

- **Reemplazar(int indice, T nuevoDato):** Modifica directamente el puntero de contenido en el nodo correspondiente sin romper la cadena de enlaces.

2. Cola Genérica FIFO (**Cola<T>**):

Implementada mediante enlaces simples con punteros explícitos a la cabeza (**frente**) y al final (**atrás**). Permite operaciones de tiempo constante $O(1)$ para **Encolar(T)** y **Desencolar()**. Esta estructura constituye el motor conceptual para la administración de la frontera de exploración en los algoritmos de búsqueda en anchura.

3. Matriz Ortogonal Dinámica (**MatrizOrtogonal**):

Para modelar la malla bidimensional de la ciudad sin recurrir a arreglos multidimensionales estándar ($T[,]$ o $T[][]$), se construyó una estructura cuadrilátera de nodos vinculados de cuatro vías (**NodoMatriz**). Cada nodo sostiene una referencia a su objeto **Celda** y cuatro punteros direccionales: **Arriba**, **Abajo**, **Izquierda** y **Derecha**.

C#

```
public class NodoMatriz {
    public Celda Dato { get; set; }
    public NodoMatriz Arriba { get; set; }
    public NodoMatriz Abajo { get; set; }
    public NodoMatriz Izquierda { get; set; }
    public NodoMatriz Derecha { get; set; }
    public bool Visitado { get; set; }
    public NodoMatriz Padre { get; set; }
}
```

Esta implementación de la matriz ortogonal permite la navegación en tiempo constante entre celdas contiguas mediante la desreferenciación directa de punteros, garantizando una representación rigurosamente orientada a objetos de la topología espacial.

2.4. Lógica de Negocio y Algoritmos de Búsqueda de Caminos (BFS)

El núcleo algorítmico reside en la clase **BuscadorCaminos**. La naturaleza no ponderada (o ponderada por estado) del espacio de estados hace que la Búsqueda en Anchura (BFS) sea el algoritmo idóneo para garantizar el hallazgo de rutas de longitud mínima. Sin embargo, la semántica operacional exige dos variantes algorítmicas marcadamente distintas:

A. Búsqueda de Camino de Rescate (BFS Clásico Multi-Origen)

Para misiones de evacuación civil, el robot **ChapinRescue** puede ingresar por cualquiera de los puntos de entrada existentes en la ciudad. El algoritmo inicializa la **Cola<NodoMatriz>** encolando simultáneamente *todos* los nodos marcados como **Entrada**, estableciendo sus marcas de **Visitado = true**. Esto equivale a crear un nodo fuente virtual común que expande frentes de onda en paralelo desde todas las entradas.

En cada iteración, se desencola un nodo y se inspeccionan sus cuatro vecinos directos (Arriba, Abajo, Izquierda, Derecha). Un vecino es transitable si y solo si:

1. No ha sido visitado previamente (**!vecino.Visitado**).

2. Su tipo de celda es **Camino** o es exactamente la celda **Civil** objetivo.

Se prohíbe explícitamente el paso por celdas **Intransitable**, **Militar** o **Recurso**. Cuando el nodo extraído coincide con la coordenada del civil, se detiene la exploración y se reconstruye la ruta recorriendo en sentido inverso los punteros **Padre** acumulados.

B. Búsqueda de Camino de Extracción (BFS con Estado y Poda de Capacidad)

El cálculo de ruta para **ChapinFighter** introduce un problema fundamental: el espacio de estados no se limita al par de coordenadas \$(fila, columna)\$, sino a la terna \$(fila, columna, capacidadRemanente)\$. Esto se debe a que visitar una celda con una capacidad militar alta destruye recursos del robot, imposibilitando superar combates posteriores; en cambio, alcanzar la misma celda por un camino alternativo conservando mayor capacidad de combate puede habilitar rutas que antes eran inviables.

Por consiguiente, el algoritmo no puede simplemente descartar celdas posicionalmente visitadas. Se definió la estructura auxiliar **EstadoBusqueda**:

```
C#
public class EstadoBusqueda {
    public NodoMatriz Nodo { get; set; }
    public int CapacidadRestante { get; set; }
    public EstadoBusqueda PadreEstado { get; set; }
}
```

Para controlar la explosión combinatoria y evitar ciclos infinitos, se implementa un mecanismo de **poda por dominancia de estado**. Se mantiene una matriz auxiliar memoria del mismo tamaño de la malla, **mejorCapacidadAlcanzada**[fila, columna], inicializada en -1. Un nodo vecino puede ser encolado bajo dos condiciones alternativas:

- Si la celda es transitable (Camino o Recurso Objetivo).
- Si la celda contiene una **UnidadMilitar** con capacidad \$C_U\$, siempre que la capacidad del robot \$C_R\$ cumpla \$C_R > C_U\$, calculando la nueva capacidad como \$C'_R = C_R - C_U\$.

Un estado candidato solo se acepta e inserta en la **Cola**<EstadoBusqueda> si su capacidad remanente es estrictamente mayor que el valor previamente registrado en la matriz auxiliar (\$C'_R > mejorCapacidadAlcanzada[fila, columna]\$). De cumplirse, se actualiza dicho registro y se procede con la expansión. Al alcanzar el recurso objetivo, la ruta completa (incluyendo los decrementos de combate) se reconstruye navegando la referencia **PadreEstado**.

2.5. Integración con Graphviz para la Generación de Reportes Gráficos

La presentación gráfica de la malla y el camino calculado se resuelve mediante la interacción con el motor de renderizado Graphviz. La clase **GeneradorGraphviz** procesa la **MatrizOrtogonal** y genera programáticamente un archivo de especificación en lenguaje DOT.

Para garantizar una disposición rigurosamente cuadrilátera que respete la topología de la ciudad, se utiliza el motor layout **neato** fijando las posiciones absolutas de cada nodo

mediante el atributo `pos="columna,-fila!"`. Cada celda se renderiza como un nodo de forma cuadrada (`shape=record` o `square`) codificado por colores según la siguiente convención cromática:

- **Negro (#000000):** Celda Intransitable (*).
- **Verde (#2ECC71):** Punto de Entrada (E).
- **Blanco (#FFFFFF):** Camino libre ().
- **Rojo (#E74C3C):** Unidad Militar (con etiqueta indicando su capacidad de combate).
- **Azul (#3498DB):** Unidad Civil (C).
- **Gris (#95A5A6):** Recurso estratégico (R).
- **Naranja (#E67E22):** Celdas pertenecientes a la ruta óptima calculada.

Una vez redactado el archivo `.dot` en disco, la clase invoca al proceso del sistema operativo mediante `System.Diagnostics.Process` pasando los argumentos `-Tpng mapa.dot -o mapa.png`. El archivo de imagen resultante es cargado dinámicamente en el componente `Image` de la interfaz WPF, brindando al usuario una retroalimentación visual clara e inmediata.

III. CONCLUSIONES

1. El desarrollo de Tipos de Datos Abstractos propios a nivel de memoria dinámica (punteros en Heap) constituye una práctica indispensable para comprender los fundamentos teóricos de la ciencia de la computación. Al prescindir de las colecciones integradas de .NET, se adquiere un dominio riguroso sobre el control de referencias, la complejidad temporal y espacial de las estructuras de datos, y el impacto de la gestión de memoria en el rendimiento del software.
2. La abstracción formal mediante Programación Orientada a Objetos permite modelar dominios complejos de manera extensible y mantenible. La separación explícita de responsabilidades en capas (Modelos, TDA y Servicios) facilitó la incorporación de reglas de negocio altamente diferenciadas —como la gestión de combates o el procesamiento XML— sin comprometer la estabilidad global del sistema.
3. La inclusión de atributos de estado dinámicos en los algoritmos de recorrido sobre grafos transforma sustancialmente la naturaleza de la búsqueda. Mientras que un BFS clásico basta para encontrar rutas cortas en espacios estáticos (rescate civil), el problema de extracción demostró que cuando el estado del agente evoluciona en función del camino recorrido, es imperativo extender el espacio de estados a combinaciones posicionales-cuantitativas y aplicar técnicas de poda por dominancia para evitar la sobrecarga computacional.